

3.4 模式收益和成本模型

静态模式收益和动态模式收益

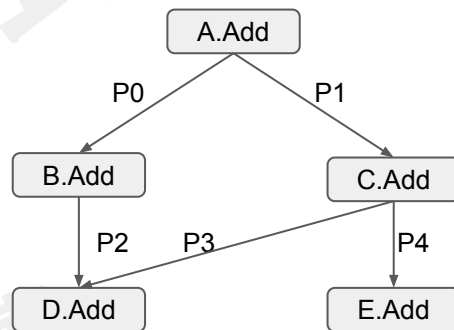
模式收益是重写模式的固有属性。

```
class Pattern {  
    ...  
    const PatternBenefit benefit;  
    ...  
};
```

```
class PatternBenefit {  
    ...  
private:  
    unsigned short representation{ImpossibleToMatchSentinel};  
};
```

模式收益可以为操作合法化过程的决策机制提供最优模式选择依据。

- 静态模式收益: patterns.add 接口参数中指定
- 动态模式收益: OperationLegalizer 类构造函数根据成本模型的分析结果计算得到



构建合法化图

```
OperationLegalizer::OperationLegalizer(const ConversionTarget &targetInfo,  
                                       const FrozenRewritePatternSet &patterns)  
    : target(targetInfo), applicator(patterns) {  
    DenseMap<OperationName, LegalizationPatterns> legalizerPatterns;  
    LegalizationPatterns anyOpLegalizerPatterns;  
  
    buildLegalizationGraph(anyOpLegalizerPatterns, legalizerPatterns);  
    computeLegalizationGraphBenefit(anyOpLegalizerPatterns, legalizerPatterns);  
}
```

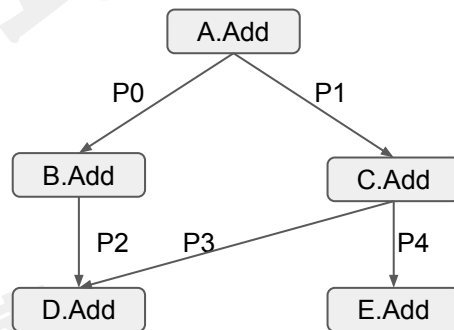
初始条件下, buildLegalizationGraph() 函数首先构造以下映射集合和列表:

```
parentOps: { B.Add : (A.Add), C.Add : (A.Add), D.Add : (B.Add, C.Add), E.Add : (C.Add) }  
invalidPatterns : { A.Add : (P0, P1), B.Add : (P2), C.Add : (P3, P4) }  
patternWorklist : {P0, P1, P2, P3, P4}
```

遍历 patternWorklist, 填充 **legalizerPatterns** 如下:

```
P4: C.Add -> E.Add, E.Add 合法 => legalizerPatterns[C.Add] : {P4}, 删除 invalidPatterns[C.Add] : {P4}  
P3: C.Add -> D.Add, D.Add 合法 => legalizerPatterns[C.Add] : {P3}, 删除 invalidPatterns[C.Add] : {P3}  
P2: B.Add -> D.Add, D.Add 合法 => legalizerPatterns[B.Add] : {P2}, 删除 invalidPatterns[B.Add] : {P2}  
P1: A.Add -> C.Add, C.Add 合法 => legalizerPatterns[A.Add] : {P1}, 删除 invalidPatterns[A.Add] : {P1}  
P0: A.Add -> B.Add, B.Add 合法 => legalizerPatterns[A.Add] : {P0}, 删除 invalidPatterns[A.Add] : {P0}
```

```
legalizerPatterns:  
{ C.Add : (P4, P3), B.Add : (P2), A.Add : (P1, P0) }
```



计算操作合法化深度

合法化深度可分为模式合法化深度和操作合法化深度两类。

- 模式合法化深度
- 操作合法化深度

```
void OperationLegalizer::computeLegalizationGraphBenefit(
    LegalizationPatterns &anyOpLegalizerPatterns,
    DenseMap<OperationName, LegalizationPatterns> &legalizerPatterns) {
    // The smallest pattern depth, when legalizing an operation.
    DenseMap<OperationName, unsigned> minOpPatternDepth;
```

```
// For each operation that is transitively legal, compute a cost for it.
```

```
for (auto &opIt : legalizerPatterns)
```

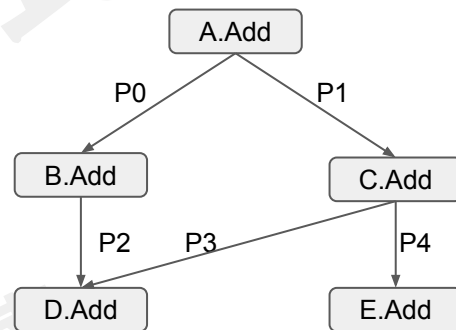
```
if (!minOpPatternDepth.count(opIt.first))
```

```
    computeOpLegalizationDepth(opIt.first, minOpPatternDepth,
                                legalizerPatterns);
```

```
...
```

```
applicator.applyCostModel([&](const Pattern &pattern) {...});
```

```
}
```



legalizerPatterns:

{ C.Add : (P4, P3), B.Add : (P2), A.Add : (P1, P0) }

计算操作合法化深度

```
unsigned OperationLegalizer::computeOpLegalizationDepth(  
    OperationName op, DenseMap<OperationName, unsigned> &minOpPatternDepth,  
    DenseMap<OperationName, LegalizationPatterns> &legalizerPatterns) {  
    auto depthIt = minOpPatternDepth.find(op);  
    if (depthIt != minOpPatternDepth.end())  
        return depthIt->second;  
    return depthIt->second;  
}
```

// If a mapping for this operation does not exist, then this operation
// is always legal. Return 0 as the depth for a directly legal operation.

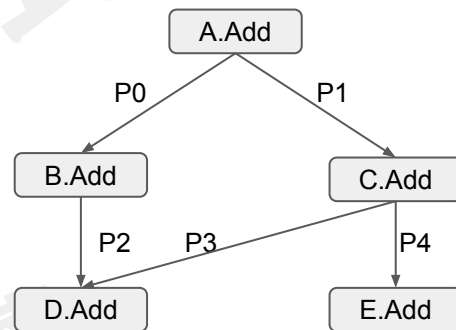
```
auto opPatternsIt = legalizerPatterns.find(op);  
if (opPatternsIt == legalizerPatterns.end() || opPatternsIt->second.empty())  
    return 0u;
```

```
minOpPatternDepth.try_emplace(op, std::numeric_limits<unsigned>::max());
```

// Apply the cost model to the operation patterns, and update the minimum
// depth.

```
unsigned minDepth = applyCostModelToPatterns(  
    opPatternsIt->second, minOpPatternDepth, legalizerPatterns);  
minOpPatternDepth[op] = minDepth;  
return minDepth;
```

```
}
```



legalizerPatterns:

```
{ C.Add : (P4, P3), B.Add : (P2), A.Add : (P1, P0) }
```

计算操作合法化深度

```
unsigned OperationLegalizer::applyCostModelToPatterns( ... ) {  
    unsigned minDepth = std::numeric_limits<unsigned>::max();
```

```
    SmallVector<std::pair<const Pattern *, unsigned>, 4> patternsByDepth;
```

```
    patternsByDepth.reserve(patterns.size());
```

```
    for (const Pattern *pattern : patterns) {
```

```
        unsigned depth = 1;
```

```
        for (auto generatedOp : pattern->getGeneratedOps()) {
```

```
            unsigned generatedOpDepth = computeOpLegalizationDepth(
```

```
                generatedOp, minOpPatternDepth, legalizerPatterns);
```

```
            depth = std::max(depth, generatedOpDepth + 1);
```

```
        }
```

```
        patternsByDepth.emplace_back(pattern, depth);
```

```
        minDepth = std::min(minDepth, depth);
```

```
    }
```

```
    std::stable_sort(...);
```

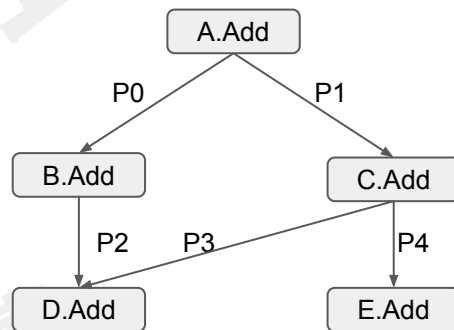
```
    patterns.clear();
```

```
    for (auto &patternIt : patternsByDepth)
```

```
        patterns.push_back(patternIt.first);
```

```
    return minDepth;
```

```
}
```



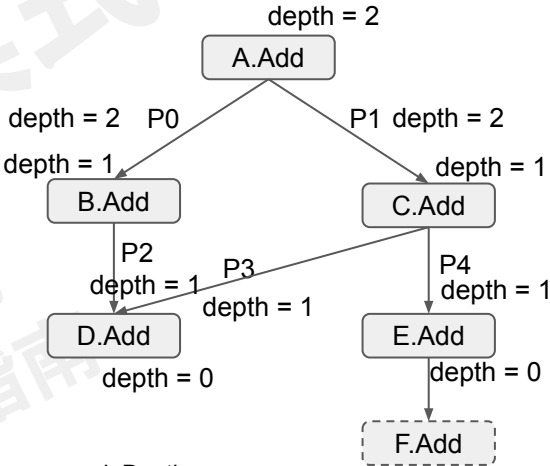
legalizerPatterns:

{ C.Add : (P4, P3), B.Add : (P2), A.Add : (P1, P0) }

计算操作合法化深度

已知有效模式列表 legalizerPatterns:
{ C.Add : (P4, P3), B.Add : (P2), A.Add : (P1, P0) }

调用 computeLegalizationGraphBenefit 函数计算合法化深度和 动态模式收益值



Op	minOpPatternDepth	generatedOp	generatedOpDepth	patternsByDepth	minDepth	
C.Add	minOpPatternDepth[C.Add] = max	E.Add	0 (E.Add 合法)	patternsByDepth[P4] = max(1, 0+1)	1	两个 pattern 深度相同, 按收益排序
	minOpPatternDepth[C.Add] = max->1	D.Add	0 (D.Add 合法)	patternsByDepth[P3] = max(1, 0+1)	1	
B.Add	minOpPatternDepth[B.Add] = max->1	D.Add	0 (D.Add 合法)	patternsByDepth[P2] = max(1, 0+1)	1	只有一个 pattern, 无需排序
A.Add	minOpPatternDepth[A.Add] = max	C.Add	1 (minDepth)	patternsByDepth[P1] = max(1, 1+1)	2	两个 pattern 深度相同, 按收益排序
	minOpPatternDepth[A.Add] = max->2	B.Add	1 (minDepth)	patternsByDepth[P0] = max(1, 1+1)	2	

{ C.Add : (P4, P3), B.Add : (P2), A.Add : (P1, P0) }

不同深度内, 按深度升序排序

相同深度内, 按静态模式收益降序排序

成本模型的实现

```
using CostModel = function_ref<PatternBenefit(const Pattern &);>;
```

```
void OperationLegalizer::computeLegalizationGraphBenefit(
    LegalizationPatterns &anyOpLegalizerPatterns,
    DenseMap<OperationName, LegalizationPatterns> &legalizerPatterns) {
    ...
    for (auto &oplt : legalizerPatterns)
        if (!minOpPatternDepth.count(oplt.first))
            computeOpLegalizationDepth(oplt.first, minOpPatternDepth,
                                         legalizerPatterns);
    ...
    applicator.applyCostModel([&](const Pattern &pattern) {
        ArrayRef<const Pattern *> orderedPatternList;
        if (std::optional<OperationName> rootName = pattern.getRootKind())
            orderedPatternList = legalizerPatterns[*rootName];
        else
            orderedPatternList = anyOpLegalizerPatterns;

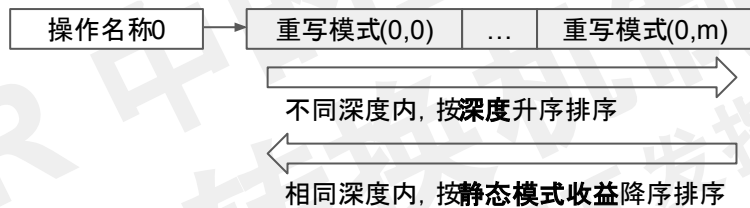
        auto *it = llvm::find(orderedPatternList, &pattern);
        if (it == orderedPatternList.end())
            return PatternBenefit::impossibleToMatch();

        return PatternBenefit(std::distance(it, orderedPatternList.end()));
    });
}
```

```
void PatternApplicator::applyCostModel(CostModel model) {
    ...
    llvm::SmallDenseMap<const Pattern *, PatternBenefit> benefits;
    auto cmp = [&benefits](const Pattern *lhs, const Pattern *rhs) {
        return benefits[lhs] > benefits[rhs];
    };
    auto processPatternList =
        [&](SmallVectorImpl<const RewritePattern *> &list) {
            ...
            // Collect the dynamic benefits for the current pattern list.
            benefits.clear();
            for (const Pattern *pat : list)
                benefits.try_emplace(pat, model(*pat));

            std::stable_sort(list.begin(), list.end(), cmp);
            ...
        };
    for (auto &it : patterns)
        processPatternList(it.second);
    processPatternList(anyOpPatterns);
}
```


重写模式排序规则



模式排序过程总结

